

To analyze this scenario from the perspective of an **Executive Director (ED) on the Cash Equities Central Risk Book (CRB) desk at Nomura**, we must first ground the problem in our operational reality.

At Nomura, the CRB acts as an internal aggregator of residual risk generated by client facilitation and market-making. When we internalize risk, we cannot simply dump it into the market without causing massive market impact, which erodes our corporate P&L. We require optimal execution schedules. However, testing a new execution policy ( $\pi_e$ ) live is incredibly expensive and risky. We must rely on **Offline Policy Evaluation (OPE)** using historical trade logs generated by our legacy behavior policies ( $\pi_b$ ).

## 1. The Mathematical Deep Dive: Step-by-Painstaking-Step

In a technical interview, an exceptional candidate must deconstruct the mathematics of counterfactual estimation from first principles. Here is the exact step-by-step mathematical derivation of how we correct for selection bias in execution logs.

### Step 1: Formalizing the Execution Environment as a MDP

We define our system at any discrete time step  $t$  using the following components:

- **State** ( $s \in \mathcal{S}$ ): A vector of market features:  $\mathbf{s} = [\text{time-of-day, bid-ask spread, rolling volatility, order size, Average Daily Volume (ADV)}]$ .
- **Action** ( $a \in \mathcal{A}$ ): The chosen execution rate  $v(t)$ , which dictates how aggressively we trade.
- **Reward** ( $r$ ): The negative Implementation Shortfall ( $-IS$ ), where our goal is to maximize reward (minimize trading costs).

### Step 2: The Core Problem—Selection Bias and the Feedback Loop

Our historical data consists of trajectories sampled from a legacy execution algorithm, which acts as our **behavior policy**  $\pi_b(a|s)$ . The true expected value (performance) of a new target policy  $\pi_e$  is defined as the expected reward under its own state-action distribution:

$$V(\pi_e) = \mathbb{E}_{s \sim d^{\pi_e}, a \sim \pi_e} [r(s, a)]$$

Because we only have data generated by  $a \sim \pi_b$ , if we naively evaluate  $\pi_e$  using the historical rewards, our estimate will be completely biased. The history tells us what happened when  $\pi_b$  chose action  $a$ , but it doesn't tell us what would have happened if  $\pi_e$  chose a different action  $a'$ .

### Step 3: Deriving the Inverse Propensity Score (IPS / IPW) Weight

To evaluate  $\pi_e$  without running it live, we apply Change of Variables (Importance Sampling) to shift the expectation from  $\pi_e$  to  $\pi_b$ :

$$V(\pi_e) = \int_{\mathcal{S}} \int_{\mathcal{A}} r(s, a) \pi_e(a|s) d^{\pi_e}(s) da ds$$

Assuming the state distribution mismatch between the two policies is negligible over short horizons ( $d^{\pi_e}(s) \approx d^{\pi_b}(s)$ ), we can rewrite this as:

$$V(\pi_e) \approx \int_{\mathcal{S}} \int_{\mathcal{A}} r(s, a) \pi_e(a|s) d\pi_b(s) da ds$$

Now, we multiply and divide the integrand by the known behavior policy probability  $\pi_b(a|s)$ :

$$V(\pi_e) = \int_{\mathcal{S}} \int_{\mathcal{A}} r(s, a) \frac{\pi_e(a|s)}{\pi_b(a|s)} \pi_b(a|s) d\pi_b(s) da ds$$

This transforms the expression back into an expectation over our historically observed data distribution (under  $\pi_b$ ):

$$V(\pi_e) = \mathbb{E}_{s \sim d^{\pi_b}, a \sim \pi_b} \left[ \frac{\pi_e(a|s)}{\pi_b(a|s)} r(s, a) \right]$$

Replacing the theoretical expectation with its empirical sample mean over  $n$  historical observations yields the standard **IPW Estimator**:

$$\hat{V}^{IPW}(\pi_e) = \frac{1}{n} \sum_{i=1}^n \frac{\pi_e(a_i|s_i)}{\pi_b(a_i|s_i)} r_i$$

#### Step 4: Variance Breakdown and Propensity Explosion

While  $\hat{V}^{IPW}$  is an unbiased estimator, its variance can be massive. Let the importance weight be  $w_i = \frac{\pi_e(a_i|s_i)}{\pi_b(a_i|s_i)}$ . If our new policy proposes an execution rate that the old policy rarely tried,  $\pi_b(a_i|s_i) \rightarrow 0$ . This causes  $w_i \rightarrow \infty$ , blowing up the variance of our estimator:

$$\text{Var}(\hat{V}^{IPW}) \propto \mathbb{E}_{\pi_b} \left[ \left( \frac{\pi_e(a|s)}{\pi_b(a|s)} \right)^2 r(s, a)^2 \right]$$

#### Step 5: Deriving the Doubly-Robust (DR) Estimator

To fix this variance explosion, we introduce a baseline model—a Gradient Boosting Machine (GBM)—to act as a **Direct Method (DM)** estimator. The GBM fits a function  $\hat{Q}(s, a)$  to predict the negative shortfall ( $r$ ) given the market state and execution rate:

$$\hat{Q}(s, a) \approx \mathbb{E}[r|s, a]$$

The Doubly-Robust estimator combines the GBM prediction with the IPW correction using a control-variate design:

$$\hat{V}^{DR}(\pi_e) = \frac{1}{n} \sum_{i=1}^n \left[ \mathbb{E}_{a \sim \pi_e} [\hat{Q}(s_i, a)] + \frac{\pi_e(a_i|s_i)}{\pi_b(a_i|s_i)} (r_i - \hat{Q}(s_i, a_i)) \right]$$

Where:

1.  $\mathbb{E}_{a \sim \pi_e} [\hat{Q}(s_i, a)]$ : Is the baseline value if we trusted our machine learning model completely.
2.  $r_i - \hat{Q}(s_i, a_i)$ : Is the residual error of our GBM model on the realized trade.
3.  $\frac{\pi_e(a_i|s_i)}{\pi_b(a_i|s_i)}$ : Scales that residual error to ensure the final metric remains statistically unbiased.

If our GBM model  $\hat{Q}$  is highly accurate, the residual error is close to zero, and the variance of the entire expression drops significantly. If our propensity score is wrong but the GBM is perfect, or if the GBM is

completely wrong but the propensity score is perfect, the estimator remains unbiased—hence the term **Doubly Robust**.

## 2. Application of Theory and Production-Grade Implementation

An elite candidate cannot just write formulas on a whiteboard; they must translate this math into stable, production-ready code that handles real-world edge cases like propensity clipping and matrix evaluation.

### Production-Grade Python Implementation

Below is the complete framework for implementing the Doubly-Robust Off-Policy Evaluation system. It leverages a trained Gradient Boosting regressor to build the  $\hat{Q}(s, a)$  direct model and applies a stabilized propensity ratio evaluation.

```
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor

class CashEquitiesOPEvaluator:
    """
    Production-grade Doubly-Robust Evaluation engine for optimizing Central Risk
    Book
    execution policies without live market deployment risk.
    """
    def __init__(self, min_propensity: float = 0.01, max_weight: float = 20.0):
        self.min_propensity = min_propensity
        self.max_weight = max_weight
        self.q_model = GradientBoostingRegressor(
            n_estimators=150,
            learning_rate=0.05,
            max_depth=5,
            random_state=42
        )

    def _clip_propensities(self, propensities: np.ndarray) -> np.ndarray:
        """Clips behavior propensities to prevent division-by-zero and exploding
        variance."""
        return np.clip(propensities, self.min_propensity, 1.0)

    def train_direct_model(self, states: np.ndarray, actions: np.ndarray, rewards:
np.ndarray):
        """
        Trains the Q-Model (GBM) to map (State, Action) -> Reward (Negative
        Shortfall).
        """
        # Concatenate states and actions as feature inputs for the GBM model
        X = np.hstack((states, actions.reshape(-1, 1)))
        self.q_model.fit(X, rewards)

    def evaluate_policy(self,
        states: np.ndarray,
```

```

        actions_logged: np.ndarray,
        rewards_logged: np.ndarray,
        pi_b_probs: np.ndarray,
        target_policy_callback) -> dict:
    """
    Computes Direct Method, IPW, and Doubly-Robust values for a target
    execution policy.

    Parameters:
    -----
    states : np.ndarray, shape (n_samples, n_features)
    actions_logged : np.ndarray, shape (n_samples,) - historical execution
    rates chosen by pi_b
    rewards_logged : np.ndarray, shape (n_samples,) - historical realized
    negative shortfall
    pi_b_probs : np.ndarray, shape (n_samples,) - probability density of pi_b
    choosing actions_logged
    target_policy_callback : function - returns (action, probability) under
    the new execution strategy
    """
    n_samples = states.shape[0]

    # 1. Evaluate target policy behavior on historical states
    pi_e_actions = np.zeros(n_samples)
    pi_e_probs = np.zeros(n_samples)
    for i in range(n_samples):
        act, prob = target_policy_callback(states[i])
        pi_e_actions[i] = act
        pi_e_probs[i] = prob

    # 2. Compute Propensity Ratios with structural clipping rules
    pi_b_probs_clipped = self._clip_propensities(pi_b_probs)
    importance_weights = pi_e_probs / pi_b_probs_clipped
    importance_weights = np.clip(importance_weights, 0, self.max_weight)

    # 3. Predict Q(s_i, a_logged) and Q(s_i, a_target) using our Direct Model
    X_logged = np.hstack((states, actions_logged.reshape(-1, 1)))
    X_target = np.hstack((states, pi_e_actions.reshape(-1, 1)))

    q_pred_logged = self.q_model.predict(X_logged)
    q_pred_target = self.q_model.predict(X_target)

    # 4. Compute Estimators
    # Direct Method (DM) Value
    v_dm = np.mean(q_pred_target)

    # Inverse Propensity Weighting (IPW) Value
    v_ipw = np.mean(importance_weights * rewards_logged)

    # Doubly-Robust (DR) Value Calculation
    dr_elements = q_pred_target + importance_weights * (rewards_logged -
    q_pred_logged)
    v_dr = np.mean(dr_elements)
    dr_variance = np.var(dr_elements) / n_samples

```

```

return {
    "v_dm": v_dm,
    "v_ipw": v_ipw,
    "v_dr": v_dr,
    "dr_std_error": np.sqrt(dr_variance)
}

```

### 3. Executive Director's Follow-up Questions & Deep-Dive Countermeasures

If you present this to an ED on the desk, they will instantly pressure test your implementation's vulnerability to real-world continuous actions and tail risks.

Follow-Up 1: "In continuous action spaces (execution rates  $v \in [0, 1]$ ), the probability density  $\pi(a|s)$  can drop to near zero or become highly unstable. How do you handle this structurally in production?"

- **The Fix:** We cannot use raw continuous densities safely in the denominator of an IPW calculation. In our production systems, we resolve this in one of two ways:
  1. **Action Discretization:** We map execution rates into structured categorical buckets (e.g., 5%, 10%, 20% of ADV) and use a categorical distribution output via GBDT classifiers or soft-max heads.
  2. **Gaussian Policy Mapping:** If we keep actions continuous, we assume both the behavior policy and target policy follow Gaussian profiles  $\mathcal{N}(\mu(s), \sigma^2)$ . We then compute the continuous probability density function (PDF) value for both at the logged action point  $a_i$  and clamp the resulting weight ratio strictly using:

```
importance_weights = np.clip(pi_e_pdf / pi_b_pdf, 0.0, max_weight)
```

Follow-Up 2: "Your GBM model  $\hat{Q}(s, a)$  might predict execution costs accurately for average market conditions, but it will fail entirely during tail events (e.g., sudden market regime shifts or a liquidity drain). How does this affect your DR estimator?"

- **The Fix:** Because our DR implementation utilizes a control variate architecture, it degrades gracefully. During a tail event where our GBM model fails catastrophically ( $\hat{Q}(s_i, a_i) \rightarrow \text{garbage}$ ), the residual term  $(r_i - \hat{Q}(s_i, a_i))$  acts as a correction mechanism. As long as our logged behavior propensity  $\pi_b$  remains accurate (which we control, since it is *our* legacy algorithm), the importance weight multiplication maps the metric back to an unbiased representation.
- To further robustify the system against tail-driven variance explosions, we deploy a **Self-Normalized Doubly Robust (SNDP)** estimator, ensuring that the importance weights sum strictly to 1:

$$\hat{V}^{SND R}(\pi_e) = \sum_{i=1}^n \left[ \frac{\frac{\pi_e(a_i|s_i)}{\pi_b(a_i|s_i)}}{\sum_{j=1}^n \frac{\pi_e(a_j|s_j)}{\pi_b(a_j|s_j)}} \left( r_i - \hat{Q}(s_i, a_i) \right) \right] + \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{a \sim \pi_e} [\hat{Q}(s_i, a)]$$